

XQDot4Z: Quantized Dot Products on a RISC-V Processor

Steve Andy Ildefonso Santos
Computer Science student
Working draft v0.10 — not for submission

Abstract—Low-bit-weight storage reduces data size, but processing these weights may require extraction, conversion, and correction of the numerical origin. This work proposes studying an operation that combines four products of unsigned four-bit weights and signed eight-bit activations with zero-point correction. The question is when this fusion is useful compared with scalar multiplication and with a packed dot product followed by separate correction. A teaching-oriented thirty-two-bit processor is proposed as the platform, first to verify the operation and then to evaluate dot products and matrix–vector multiplication. This document is an initial draft: it presents the question and reports a registered campaign of 2280 simulated cases. Fusion reduces cycles against both alternatives, by a margin that narrows as the row count grows and that vanishes when the zero point is zero. With a varying zero point the advantage is paid for in code size, because the immediate forces specialization. These are simulated cycles: not physical time, energy or area.

Index Terms—quantized inference, low-precision arithmetic, instruction-set extensions, hardware verification

I. INTRODUCTION

Neural network layers perform products and sums involving weights and activations. Quantization allows part of this computation to be represented using integers and scale parameters [1]. However, storing more weights per word does not eliminate the work needed to consume them on a processor. Field extraction and operand preparation can be relevant costs.

We propose studying XQDot4Z on Kuntur, a teaching-oriented RISC-V processor with a five-stage pipeline. The aim is to investigate a bounded operation and its costs, rather than to develop a complete accelerator or execute a large language model at this stage.

II. BACKGROUND AND RELATED WORK

In affine quantization, an integer code q approximately represents $r = s(q - z)$, where $s > 0$ is the scale and z the zero-point. With activations whose zero-point is zero, the arithmetic kernel under study is

$$d = \sum_{i=0}^3 (q_{w,i} - z_w) q_{a,i}. \quad (1)$$

Weights are stored as U4 (unsigned four-bit integers) and activations as S8 (signed eight-bit integers). The proposed output is an INT32 subtotal (a signed 32-bit integer); scales, bias, and accumulation of other subtotals remain outside this operation.

XpulpNN and Dustin provide precedents for low-precision instructions and mixed-precision arithmetic on RISC-V [2],

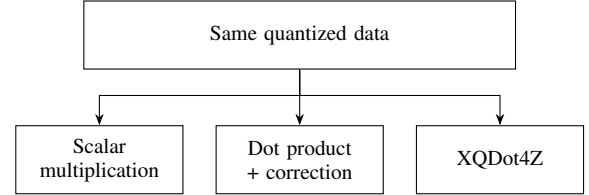


Fig. 1. Proposed comparison on a common baseline. This diagram does not represent measured results.

[3]. Low-bit dot products are therefore not treated as a new concept. The specific contribution must be established through a more detailed comparison with prior work and the project’s own results.

An important alternative to fusion is

$$d = \sum_i q_{w,i} q_{a,i} - z_w \sum_i q_{a,i}. \quad (2)$$

The activation sum can be reused across matrix rows. A valid comparison must allow this optimization.

III. RESEARCH QUESTION AND SCOPE

Under what conditions does fusing zero-point correction into a U4-by-S8 dot product reduce execution cost compared with scalar multiplication and factored correction, accounting for activation reuse and zero-point delivery?

The initial scope is one core, batch size one, and small dot-product and matrix–vector kernels. The first phase will evaluate correctness and simulation cycles. Synthesis and board-level evaluation are later stages. Energy or physical execution time will not be inferred solely from instruction or cycle counts.

IV. PROPOSED METHOD

A. Platform and Verification

The original processor is preserved, and development takes place in an independent clone. Core corrections are verified separately and will form a common baseline for the variants. Unimplemented instructions must be detected, without claiming full conformance to an instruction set that the processor does not yet cover.

The v0.1 numerical contract defines a 32-bit word W containing eight U4 weights and a word A containing four S8 activations. A selector h chooses the lower or upper four weights of W , without changing the activations. Index zero

TABLE I
B3 CYCLES DIVIDED BY D CYCLES, BY REGIME AND ROW COUNT. RUN
20260911T040655320069Z, 2280 CASES; REGIMES ARE NEVER
POOLED.

N	ZC $z=0$	ZC $z=8$	ZC $z=15$	ZS
1	1.00	1.86	1.88	1.00
4	1.00	1.25	1.26	1.46
16	1.00	1.08	1.08	1.28

occupies the least significant bits, and z_w is shared by all four terms. The subtotal lies in $[-7680, 7680]$ and is represented in 32-bit two’s complement, without implicit accumulation.

A Python reference using unbounded integers is available, checked through examples, packing tests, exhaustive single-term tuples, and random dot products compared with (2). The isolated RTL unit is combinational, with four lanes, S13 products, and an adder tree with explicit sign extension. It is verified in Icarus Verilog and Verilator. The experimental XQDot4Zi interface optionally integrates it into Kuntur, using two source registers and immediate z_w, h . It reuses forwarding and carries metadata with the instruction. It adds no dedicated wait in E, but physical timing has not been checked. Runtime-loaded zero-points remain unresolved; an immediate cannot replace them at no cost.

The baseline includes optional scalar MUL, independent of XQDot4Zi, returning the low 32 bits of the product [4]. It does not implement full M or Zmmul. It is combinational, reuses forwarding, and is cancelled by flush/reset; functional availability does not establish physical timing.

The packed comparator (B3) has a four-lane unit without zero-point subtraction, using the same packing and selector as the fused XQDot4Z variant (D). Its subtotal lies in $[-7680, 7620]$; the correction in (2) is outside this unit. The optional XQDot4 instruction integrates it using two source GPRs, immediate h , and the same forwarding. With weights set to one, it also computes activation sums that can be reused across rows.

B. Planned Comparison

D will be compared with scalar MUL (B2) and, centrally, B3 with the correction in (2) (Fig. 1). B1 provides context without MUL: it will use bounded-width masked bit decomposition, with its policy and code frozen before the pilot. It does not represent optimal software. All variants will share data and core corrections. Loads, loops, correction, and metadata will be counted in addition to the arithmetic kernel.

The initial manifest sweeps row counts with one group of 32 elements and separates constant controls from U4 codes balanced across phases. It distinguishes zero, one, other powers of two, and remaining codes; mixed-program times are not apportioned to these strata by frequency. It evaluates static specialization, without extrapolation to dynamic metadata. Unrolling, register allocation, scheduling, and correction reuse are fixed before measurement and shared by all four variants. Under that schedule B3 corrects with one subtraction per row when z repeats and with a multiply and a subtraction when

it varies: the difference comes from policy and schedule, not architecture. All cases, including failures and slowdowns, will be reported. All four variants now have verified kernels: row traversal is derived from each encoding, and only D is forced to emit one row body per row when the zero point varies, which is reported as a required floor rather than a choice. Seeds cover inputs, not statistical repetitions; the pilot will examine control, addresses, and stalls, not only equal cycle totals.

V. RESULTS AND DISCUSSION

Isolated XQDot4Z passes 556 734 comparisons per simulator, eight negative controls, and four Icarus mutations; lint requires no waivers. Exhaustiveness only covers single-term tuples. Immediate integration passes 25 programs, including 1056 numerical operations, comparing retirement, writeback, stores, and operands, and detecting two mutations. MUL passes 86 704 vectors per simulator and 49 programs with four configurations, eight negative controls per simulator, and four Icarus mutations. Artifacts, logs, and hashes of these test suites are retained.

The registered campaign ran 2280 cases, all passing in both simulators with none missing. Cycles are identical across the ten seeds in all 228 variant, shape and setting combinations, and across the sixteen ZS phases, while the outputs differ in every one. That does not prove independence from arbitrary data, nor does it reduce the grid: the reduction gate is a deliberate revision and has not been taken.

Table I holds the central comparison. D’s advantage over B3 falls as N grows in every regime where B3 has correction to amortize, which is the trend H2 anticipated. At $z=0$ there is no advantage at all: B3 omits S_a and the correction, and the two variants coincide exactly. At $N=1$ a varying zero point is degenerate, since a single row cannot have a z that varies.

Under ZS the raw gap mixes two things. D is forced to emit one row body per row because its z is an immediate, while B3 loops, so part of the difference is control rather than correction. Subtracting the structural twin, of the 164 cycles at $N=16$ some 47 belong to loop control and 117 to the correction: the corrected ratio moves from 1.38 to 1.20 rather than 1.46 to 1.28.

Against scalar MUL, D uses between 5.4 and 6.8 times fewer cycles at $N=16$, the low end at $z=0$ where D has no correction to save; B1, without a multiplier, spends between 26 and 34 times more than D. That forced specialization is not free: at $N=16$ with a varying z , D occupies 516 code words against 72 for B3, because its encoding demands sixteen row bodies. The cycle advantage and the text cost are the same interface decision.

These figures are cycles of a simulated in-order pipeline. They are not physical time, energy or area, and D’s additional hardware is not accounted for here: that belongs to synthesis. The scope is static specialization; the regime with zero points read at run time remains unresolved.

The packed unit without correction passes 67 910 vectors per simulator, with ten negative controls per simulator

and four Icarus mutations. From 1680 RTL responses, 336 row/group outputs are reconstructed across 24 matrix–vector cases, reusing activation sums across rows. In that isolated test suite, accumulation and correction run in the verifier.

XQDot4 integration passes 123 programs in both simulators, covering all eight configurations of MUL and the two packed instructions, and detects three Icarus mutations. Four paired two-row fixtures produce eight matching outputs in B3 and D. In B3, Kuntur computes one activation sum, reuses it, and applies MUL/SUB correction; D accumulates fused subtotals. These are correctness tests with zero-points known at code-generation time, not optimized benchmarks or dynamic-metadata tests.

Performance evaluation pending. These results do not establish acceleration, resource usage, physical timing, or efficient dynamic metadata delivery.

A. Threats to Validity

The shared skeleton reduces optimization bias but does not establish optimality or eliminate software–architecture interaction. The synthetic distribution does not necessarily represent a real network. With $z=0$, separate correction can be omitted without establishing equal performance. Dispatch among immediates would support dynamic z without a new encoding, but still has no implementation or measured cost. Counters require validation; RTL cycles do not establish physical timing.

VI. CONCLUSION

Fusion reduces cycles against both factored correction and scalar multiplication, but not uniformly: the benefit narrows as N grows, vanishes at $z=0$, and with a varying zero point is charged back in code text. The answer to the question is not a number but a map of conditions, which is what this work contributes.

Left outside are the regime with zero points read at run time, which this interface cannot express, and physical evidence: area, frequency and energy need synthesis and a different methodology. The fused variant’s additional hardware is not accounted for in these figures.

REFERENCES

- [1] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” arXiv:1712.05877, 2017. [Online]. Available: <https://arxiv.org/abs/1712.05877>
- [2] A. Garofalo, G. Tagliavini, F. Conti, L. Benini, and D. Rossi, “XpulpNN: Enabling energy efficient and flexible inference of quantized neural network on RISC-V based IoT end nodes,” arXiv:2011.14325, 2020. [Online]. Available: <https://arxiv.org/abs/2011.14325>
- [3] G. Ottavi, A. Garofalo, G. Tagliavini, F. Conti, A. D. Mauro, L. Benini, and D. Rossi, “Dustin: A 16-cores parallel ultra-low-power cluster with 2b-to-32b fully flexible bit-precision and vector lockstep execution mode,” arXiv:2201.08656v2, 2023. [Online]. Available: <https://arxiv.org/abs/2201.08656v2>
- [4] RISC-V International, “The RISC-V Instruction Set Manual, Volume I: M extension, version 2.0,” ISA v20260120. <https://docs.riscv.org/reference/isa/v20260120/unpriv/m-st-ext.html>, 2026, accessed: 2026-09-08.